

GROUP PROJECT REPORT

Team Members:-

DEV PATHAK (24BCE10361)

DIVYANSH VARSHNEY(24BCE11063)

PATEL KRIS NIRANJAN(24BCE11187)



VIT[®]
BHOPAL
www.vitbhopal.ac.in

MAT2002-DISCRETE MATHEMATICS AND GRAPH THEORY

VIT BHOPAL UNIVERSITY

KOTHRIKALAN, SEHORE

MADHYA PRADESH - 466114

Project Title	Discrete Math Toolkit
Class Slot	B14+B23+D21
Class ID	BL2026270100016
Academic Semester	Fall Semester 2026-27
Faculty	Dr. Suresh Dara

ABSTRACT

Discrete mathematics provides the formal language of computer science: propositional logic underpins circuit design and program verification, relations model databases and orderings, and graphs model networks of every kind. Yet these topics are usually practised with pen and paper, where building a 16-row truth table or tracing Dijkstra's algorithm by hand is slow and error-prone.

This project presents the **Discrete Math Toolkit**, an interactive web application that solves and *explains* problems from three units of the MAT2002 syllabus. The **Logic Analyzer** parses propositional formulas, generates complete truth tables, decides logical equivalence (with a counter-example when it fails), classifies each formula as a tautology, contradiction or contingency, and minimises it into DNF and CNF using the Quine–McCluskey algorithm. The **Relation Checker** tests a relation for reflexivity, symmetry, antisymmetry and transitivity, explains every failure with the exact pairs responsible, identifies equivalence relations (with their equivalence classes) and partial orders, and displays the relation matrix. The **Graph Algorithm Visualizer** draws a weighted undirected graph and computes the shortest path with Dijkstra's algorithm and the minimum spanning tree with Kruskal's algorithm, highlighting the result on the drawing.

The toolkit was first prototyped in Python with Streamlit, SymPy and NetworkX, and then re-engineered as a dependency-free Node.js application in which every algorithm — the expression parser, the logic minimiser, the graph algorithms and even the graph layout — is implemented from first principles. The final system runs offline on any machine with Node.js, and is verified by an automated suite of **52 tests**, all of which pass.

Keywords: propositional logic, truth tables, Quine–McCluskey, binary relations, partial order, equivalence relation, Dijkstra's algorithm, Kruskal's algorithm, minimum spanning tree, Node.js.

TABLE OF CONTENTS

1. Introduction.....	3
1.1 Background.....	3
1.2 Problem Statement.....	3
1.3 Objectives	3
1.4 Scope and Syllabus Mapping	4
2. Theoretical Background.....	5
2.1 Propositional Logic.....	5
2.2 Binary Relations	5
2.3 Graph Theory.....	6
3. System Design and Architecture.....	7
3.1 Development Approach.....	7
3.2 Architecture	7
3.3 Technology Stack	7
3.4 Project Structure	8
3.5 API Endpoints.....	8
4. Implementation	9
4.1 Logic Analyzer	9
4.2 Relation Checker	9
4.3 Graph Algorithm Visualizer	10
4.4 User Interface	10
5. Results and Screenshots	11
5.1 Logic Analyzer	11
5.2 Relation Checker	13
5.3 Graph Algorithm Visualizer	15
5.4 Verification of Results by Hand	17
6. Testing.....	18
7. How to Run the Project	19
7.1 Node.js Version (Recommended).....	19
7.2 Python Version (Prototype)	19
8. Conclusion and Future Scope	20
8.1 Conclusion	20
8.2 Future Scope	20
References.....	21

1. INTRODUCTION

1.1 Background

Discrete Mathematics and Graph Theory (MAT2002) introduces the mathematical structures on which computing is built. Three of its central units — statement calculus, sets and relations, and graph theory — share a common feature: their core procedures are *algorithmic*. Deciding whether two statements are equivalent, whether a relation is a partial order, or which path through a network is cheapest are all questions that can be answered mechanically, step by step.

Because they are mechanical, these procedures are ideal candidates for software. A tool that performs them instantly lets a student check their hand-worked answers, experiment with many more examples than time would otherwise allow, and — most importantly — *see why* an answer is what it is, through highlighted truth-table rows, explicitly listed violating pairs and edges drawn on a picture of the graph.

1.2 Problem Statement

To design and implement an interactive, easy-to-run software toolkit that solves standard problems from propositional logic, binary relations and weighted graphs, presents the complete working rather than only the final answer, and is validated for correctness against known results.

1.3 Objectives

- Build a **propositional logic analyzer** that generates truth tables, checks logical equivalence, classifies formulas and derives simplified normal forms.
- Build a **relation checker** that tests the four fundamental properties of a binary relation and recognises equivalence relations and partial orders.
- Build a **graph visualizer** that finds shortest paths (Dijkstra) and minimum spanning trees (Kruskal) and displays them graphically.
- Explain every result: give counter-examples, list the pairs that break a property, and highlight solutions visually.
- Make the application **simple to run locally** on any computer, with no installation problems and no internet connection required.
- Verify correctness with an **automated test suite** covering every module.

1.4 Scope and Syllabus Mapping

Table 1: Mapping of toolkit modules to the MAT2002 syllabus

Module	Syllabus unit	Concepts covered
Logic Analyzer	Unit I – Mathematical Logic / Statement Calculus	Connectives, truth tables, tautology, contradiction, contingency, logical equivalence, DNF and CNF
Relation Checker	Unit II – Sets and Relations	Reflexive, symmetric, antisymmetric and transitive relations; equivalence relations and classes; POSET; relation matrix
Graph Visualizer	Unit IV – Graph Theory	Weighted graphs, adjacency matrix, connectivity, shortest path (Dijkstra), spanning trees (Kruskal)

2. THEORETICAL BACKGROUND

2.1 Propositional Logic

A *proposition* is a statement that is either true (T) or false (F). Compound propositions are built from variables using the connectives NOT ($\neg$), AND ($\wedge$), OR ($\vee$), exclusive OR ($\oplus$), implication ($\rightarrow$) and biconditional ($\leftrightarrow$). For a formula in n variables there are exactly 2^n possible assignments of truth values, and its **truth table** lists the formula's value under each one.

- A formula is a **tautology** if it is true in every row, a **contradiction** if it is false in every row, and a **contingency** otherwise.
- Two formulas A and B are **logically equivalent** ($A \equiv B$) exactly when they agree in every row — equivalently, when $A \leftrightarrow B$ is a tautology.
- Every formula can be written in **Disjunctive Normal Form** (an OR of AND-terms) and **Conjunctive Normal Form** (an AND of OR-clauses). The *Quine–McCluskey* method finds a minimal such form by repeatedly merging terms that differ in a single variable.

Example — the implication law

$p \rightarrow q \equiv \neg p \vee q$. The only row in which $p \rightarrow q$ is false is $p = T, q = F$; this is also the only row in which $\neg p \vee q$ is false, so the two columns are identical and the formulas are equivalent.

2.2 Binary Relations

A binary relation R on a set A is a subset of $A \times A$. The toolkit tests the four defining properties directly from their definitions:

Table 2: Properties of a binary relation R on a set A

Property	Definition
Reflexive	$(a, a) \in R$ for every $a \in A$
Symmetric	$(a, b) \in R \Rightarrow (b, a) \in R$
Antisymmetric	$(a, b) \in R$ and $(b, a) \in R \Rightarrow a = b$
Transitive	$(a, b) \in R$ and $(b, c) \in R \Rightarrow (a, c) \in R$

A relation that is reflexive, symmetric and transitive is an **equivalence relation**; it partitions A into disjoint *equivalence classes*. A relation that is reflexive, antisymmetric and transitive is a **partial order**, and (A, R) is then a **partially ordered set (POSET)**. A relation on an n -element set can also be represented by an $n \times n$ Boolean matrix M , where $M[i][j] = 1$ exactly when $(i, j) \in R$.

2.3 Graph Theory

A weighted undirected graph $G = (V, E, w)$ consists of vertices V , edges E and a weight $w(e)$ on each edge. Two classical optimisation problems are solved by the toolkit.

Shortest path — Dijkstra's algorithm. Starting from a source vertex with distance 0, the algorithm repeatedly *settles* the unvisited vertex with the smallest known distance and *relaxes* each of its edges: if going through the settled vertex gives a neighbour a shorter distance, that distance is updated. The algorithm is correct only for non-negative edge weights; with a binary heap it runs in $O((V + E) \log V)$ time.

Minimum spanning tree — Kruskal's algorithm. A spanning tree connects all V vertices using exactly $V - 1$ edges and no cycles. Kruskal's algorithm sorts the edges by weight and adds each edge in turn unless it would close a cycle; cycle detection uses a *union-find* (disjoint-set) structure. The result is a spanning tree of minimum total weight, found in $O(E \log E)$ time. If the graph is disconnected, the result is a minimum spanning *forest*.

3. SYSTEM DESIGN AND ARCHITECTURE

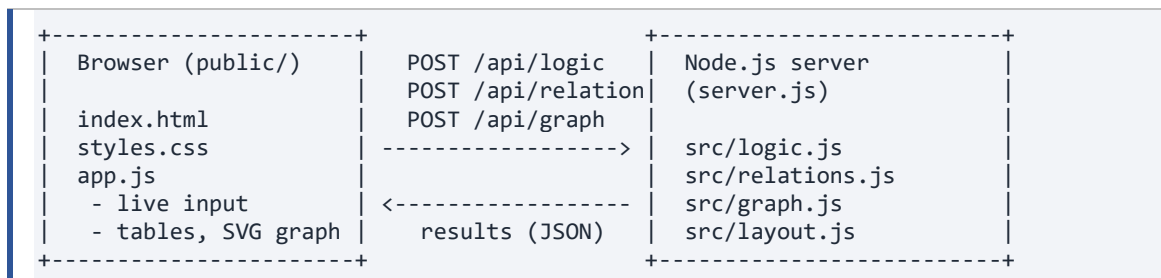
3.1 Development Approach

The toolkit was developed in two stages. A first prototype was written in **Python** using Streamlit for the interface, SymPy for symbolic logic, NetworkX for graph algorithms and Matplotlib for drawing. While functional, this version depends on several large third-party libraries, and on recent Python releases (3.13 and later) some of them do not yet publish prebuilt packages — so `pip install` can fail and the project becomes hard to run.

The project was therefore re-engineered as a **Node.js application with zero external dependencies**. Every algorithm that the Python version imported from a library was re-implemented from first principles: a tokenizer and recursive-descent parser replace SymPy, Quine–McCluskey minimisation produces the normal forms, Dijkstra and Kruskal replace NetworkX, and a deterministic force-directed layout replaces Matplotlib's graph drawing. Both versions are kept in the project and produce the same mathematical results.

3.2 Architecture

The application follows a simple client–server design. All mathematics runs on the server; the browser only collects input and renders results.



Input is sent to the server as the user types (debounced by 220 ms), so results update live without pressing a button. The server validates the input, runs the algorithms and returns a JSON object that the front end turns into tables, coloured banners and an SVG drawing of the graph.

3.3 Technology Stack

Table 3: Technology stack of the two versions

Layer	Node.js version (primary)	Python version (prototype)
Server / runtime	Node.js 18+ built-in http module	Python 3 + Streamlit
Logic engine	Hand-written tokenizer, recursive-descent parser, Quine–McCluskey	SymPy

Layer	Node.js version (primary)	Python version (prototype)
Graph algorithms	Hand-written Dijkstra (binary heap) and Kruskal (union-find)	NetworkX
Graph drawing	Force-directed layout rendered as SVG in the browser	NetworkX spring layout + Matplotlib
Interface	HTML5, CSS3 (light and dark themes), vanilla JavaScript	Streamlit widgets
Testing	node:test — 52 automated tests	Headless smoke tests (test_app.py)
External packages	None	streamlit, sympy, networkx, matplotlib, pandas

3.4 Project Structure

Table 4: Source files of the Node.js version

File	Purpose
<code>server.js</code>	HTTP server: serves the interface and the three JSON API endpoints
<code>src/logic.js</code>	Tokenizer, parser, truth-table evaluator, classifier and Quine–McCluskey DNF/CNF minimiser
<code>src/relations.js</code>	Domain and pair parsing, property checks, equivalence classes, relation matrix
<code>src/graph.js</code>	Edge-list parsing, Dijkstra, Kruskal, connected components, adjacency matrix
<code>src/layout.js</code>	Deterministic force-directed (spring) layout used to draw the graph
<code>public/</code>	Browser interface: index.html, styles.css, app.js
<code>test/</code>	52 tests: unit tests for each module plus end-to-end API tests
<code>start.bat</code> / <code>start.sh</code>	One-click launchers for Windows and macOS/Linux

3.5 API Endpoints

Table 5: REST API exposed by the server

Endpoint	Request body	Response
POST /api/logic	{ expr1, expr2 }	Parsed formulas, truth table, classification, DNF/CNF, equivalence verdict
POST /api/relation	{ domain, pairs }	Four property results with witnesses, classification, equivalence classes, matrix
POST /api/graph	{ edges, start, end }	Shortest path and cost, MST edges and weight, layout, adjacency matrix

4. IMPLEMENTATION

4.1 Logic Analyzer

Tokenizing and parsing. The expression is first split into tokens by a single regular expression; any character that is not a variable, constant, parenthesis or known operator is rejected with a clear error. Many spellings are accepted for each operator (for example $\rightarrow$, $=>$, **implies** and $\rightarrow$ all mean implication). A **recursive-descent parser** then builds an expression tree, with one function per precedence level:

```
Precedence (highest to lowest):  ~  >  &  >  ^  >  |  >  ->  >  <->

parseIff()    ->  parseImplies() ( '<->' parseImplies() )*
parseImplies() ->  parseOr() ( '->' parseImplies() )?      // right-associative
parseOr()     ->  parseXor() ( '|' parseXor() )*
parseXor()    ->  parseAnd() ( '^' parseAnd() )*
parseAnd()    ->  parseNot() ( '&' parseNot() )*
parseNot()    ->  '~' parseNot() | '(' parseIff() ')' | variable | constant
```

Because the expression is parsed by hand rather than passed to `eval()`, text typed by a user can never execute code — an important safety property for a program that runs a web server.

Truth table and verdict. The variables of both formulas are collected and sorted, all 2^n assignments are generated, and both trees are evaluated on every row. The two columns are compared row by row; if any row differs, the formulas are not equivalent and the first such row is reported as a **counter-example**. Each column is also classified as a tautology, contradiction or contingency. Input is limited to 8 variables (256 rows) to keep the table readable.

Normal forms (Quine–McCluskey). The rows where a formula is true (its *minterms*) are grouped and repeatedly merged whenever two terms differ in exactly one variable, producing the *prime implicants*. Essential prime implicants are selected first and the remaining minterms are covered greedily, giving a minimal DNF. Applying the same procedure to the false rows and complementing yields the minimal CNF.

4.2 Relation Checker

The domain and the list of ordered pairs are parsed (numbers and letters are both accepted, and every pair is checked to lie in $A \times A$). The relation is stored as a set for constant-time membership tests. Each property is then checked directly from its definition, and for every property that fails the program collects a **witness** — the exact pairs responsible:

Table 6: Witnesses reported when a property fails

Property fails	What the toolkit reports
Reflexive	Missing loops: every (a, a) not in R
Symmetric	Missing mirror pairs: every (b, a) absent although (a, b) $\in$ R

Property fails	What the toolkit reports
Antisymmetric	Violating pairs: every (a, b) , $a \neq b$, whose mirror (b, a) is also in R
Transitive	Missing pairs: every (a, c) required by some (a, b) , $(b, c) \in R$

If the relation is an equivalence relation, the toolkit also computes its **equivalence classes** by grouping each element with everything it is related to. Finally the $n \times n$ **relation matrix** is produced, with the diagonal (the loops) shaded.

4.3 Graph Algorithm Visualizer

The edge list is read one line per edge in the form **Source Target Weight**. Comment lines beginning with **#** are ignored, self-loops are dropped and a repeated edge keeps its smallest weight. The graph is then analysed by two algorithms:

```

DIJKSTRA(G, s, t)
  dist[v] = INF for all v; dist[s] = 0
  heap = {(0, s)}
  while heap not empty:
    (d, u) = heap.pop_min()
    if u already settled: continue
    settle u
    for each edge (u, v, w):
      if d + w < dist[v]:
        dist[v] = d + w; prev[v] = u
        heap.push((dist[v], v))
  follow prev[] back from t to build the path

KRUSKAL(G)
  sort edges by weight
  make-set(v) for every vertex v
  for each edge (u, v, w) in order:
    if find(u) != find(v):
      add (u, v, w) to tree
      union(u, v)
  return tree

```

Dijkstra's algorithm is disabled (with an explanation) if any weight is negative, since its correctness depends on non-negative weights; the spanning tree remains available. If the graph is disconnected the program reports a minimum spanning **forest** and says so explicitly.

Drawing the graph. Vertex positions are computed by a *force-directed* layout: connected vertices attract like springs and all vertices repel each other, and the simulation is run for 320 iterations. A fixed random seed makes the drawing identical every time the same graph is entered. The browser renders the result as an SVG image in which the start vertex, end vertex, solution edges and solution vertices are each coloured differently.

4.4 User Interface

- A sidebar switches between the three modules and shows which syllabus unit each one covers.
- Results update **live** as the user types — there is no submit button.
- One-click example chips load classic cases: De Morgan's law, the contrapositive, a tautology, the distributive law, an equivalence relation, a POSET and more.
- Colour-coded verdict banners (green for success, red for failure), highlighted mismatching truth-table rows and a shaded matrix diagonal draw attention to what matters.

5. RESULTS AND SCREENSHOTS

This chapter shows the toolkit running locally at <http://localhost:3000>. All screenshots were captured from the live application.

5.1 Logic Analyzer

Figure 1 shows the application on start-up. The sidebar on the left lists the three modules; the Logic Analyzer is open with two input expressions, $p \rightarrow q$ and $\neg p \vee q$.

Discrete Math Toolkit

Logic Analyzer
Truth tables & equivalence

Relation Checker
Properties, POSET, matrix

Graph Visualizer
Dijkstra & spanning tree

SYLLABUS COVERAGE

- Unit I: Propositional Logic
- Unit II: Sets & Relations
- Unit III: Graph Theory

Logic Analyzer Statement Calculus

Each expression is parsed into a Boolean formula. We then enumerate **all 2^n truth assignments** of its n variables and evaluate both formulas on every row. Two statements are **logically equivalent** exactly when their truth-table columns are identical in every row — that is, when $A \equiv B$ is a tautology.

Expression 1: $p \rightarrow q$

Expression 2: $\neg p \vee q$

☐ Display truth values as 1 / 0 instead of T / F [Input syntax](#)

PARSED FORMULAS

Expression 1: $\text{Implies}(p, q)$

DNF $q \vee \neg p$

CNF $\neg p \vee q$

Expression 2: $\neg p \vee q$

DNF $q \vee \neg p$

CNF $\neg p \vee q$

TRUTH TABLE (4 rows)

P	Q	EXPR 1	EXPR 2	MATCH
T	T	T	T	✓
T	F	F	F	✓
F	T	T	T	✓
F	F	T	T	✓

Rows where the two expressions disagree are highlighted.

CLASSIFICATION

EXPRESSION 1: Contingency

EXPRESSION 2: Contingency

Figure 1: Home screen of the Discrete Math Toolkit showing the Logic Analyzer

For this input the toolkit parses both formulas, derives their minimal DNF and CNF, builds the 4-row truth table and classifies each formula as a contingency. Since the two columns agree in every row, it concludes that the statements are **logically equivalent** (Figure 2) — confirming the implication law from Section 2.1.

PARSED FORMULAS

Expression 1

Implies(p, q)

DNF $q \mid \neg p$

CNF $\neg p \mid q$

Expression 2

$\neg p \mid q$

DNF $q \mid \neg p$

CNF $\neg p \mid q$

TRUTH TABLE (4 rows)

P	Q	EXPR 1	EXPR 2	MATCH
T	T	T	T	✓
T	F	F	F	✓
F	T	T	T	✓
F	F	T	T	✓

Rows where the two expressions disagree are highlighted.

CLASSIFICATION

EXPRESSION 1

Contingency (sometimes True, sometimes False)

EXPRESSION 2

Contingency (sometimes True, sometimes False)

VERDICT

✓ **The two statements are LOGICALLY EQUIVALENT.**
Their truth values agree in all 4 rows.

Figure 2: Truth table and verdict for $p \rightarrow q$ versus $\neg p \vee q$ — logically equivalent

Figure 3 compares an implication with its *converse*, $p \rightarrow q$ versus $q \rightarrow p$. The rows in which the formulas disagree are highlighted in red, and the toolkit gives a concrete **counter-example**: with $p = T$ and $q = F$, the first formula is false while the second is true.

PARSED FORMULAS

Expression 1

Implies(p, q)

DNF $q \mid \neg p$

CNF $\neg p \mid q$

Expression 2

Implies(q, p)

DNF $p \mid \neg q$

CNF $p \mid \neg q$

TRUTH TABLE (4 rows)

P	Q	EXPR 1	EXPR 2	MATCH
T	T	T	T	✓
T	F	F	T	✗
F	T	T	F	✗
F	F	T	T	✓

Rows where the two expressions disagree are highlighted.

CLASSIFICATION

EXPRESSION 1

Contingency (sometimes True, sometimes False)

EXPRESSION 2

Contingency (sometimes True, sometimes False)

VERDICT

✗ **The statements are NOT logically equivalent.**
They differ in 2 of 4 rows.
Counter-example: with $p=T$, $q=F \rightarrow$ Expr 1 = F, Expr 2 = T.

Figure 3: An implication and its converse are not equivalent — mismatching rows highlighted with a counter-example

5.2 Relation Checker

Figure 4 shows the Relation Checker module with the definitions of the four properties and the input fields for the domain and the ordered pairs.

Relation Checker Sets & Relations

A relation R on a set A is a subset of $A \times A$. We test the defining properties directly:

- **Reflexive** every a satisfies $(a, a) \in R$
- **Symmetric** $(a, b) \in R \Rightarrow (b, a) \in R$
- **Antisymmetric** $(a, b) \in R$ and $(b, a) \in R \Rightarrow a = b$
- **Transitive** $(a, b) \in R$ and $(b, c) \in R \Rightarrow (a, c) \in R$

An **equivalence relation** is reflexive + symmetric + transitive. A **partial order (POSET)** is reflexive + antisymmetric + transitive.

Domain comma-separated: 1, 2, 3, 4

Relation ordered pairs: (1,1), (2,2), (3,3), (4,4), (1,2), (2,1)

Try: ☒ Equivalence ☐ POSET (divides) ☐ Neither ☐ Empty relation

PROPERTY CHECKS — $|A| = 4, |R| = 6$

REFLEXIVE ☒ **Yes**

SYMMETRIC ☒ **Yes**

ANTISYMMETRIC ☒ **No**
Violating pairs: (1, 2), (2, 1)

TRANSITIVE ☒ **Yes**

CLASSIFICATION

☒ **R is an Equivalence Relation** (reflexive, symmetric, transitive).
It partitions the domain into these equivalence classes: {1, 2}, {3}, {4}

☒ **R is not a Partial Order (POSET).**

MATRIX REPRESENTATION

R	1	2	3	4
1	1	1	0	0
2	1	1	0	0
3	0	0	1	0
4	0	0	0	1

Entry $M[i][j] = 1$ when $(i, j) \in R$, otherwise 0. Rows are the first element, columns the second; the shaded diagonal entries are the loops.

Figure 4: The Relation Checker module

For $A = \{1, 2, 3, 4\}$ and $R = \{(1,1), (2,2), (3,3), (4,4), (1,2), (2,1)\}$ the relation is reflexive, symmetric and transitive, so it is an **equivalence relation** with classes $\{1, 2\}$, $\{3\}$ and $\{4\}$. It is not antisymmetric, and the toolkit names the violating pairs (1, 2) and (2, 1) (Figure 5).

PROPERTY CHECKS — $|A| = 4, |R| = 6$

REFLEXIVE ☒ **Yes**

SYMMETRIC ☒ **Yes**

ANTISYMMETRIC ☒ **No**
Violating pairs: (1, 2), (2, 1)

TRANSITIVE ☒ **Yes**

CLASSIFICATION

☒ **R is an Equivalence Relation** (reflexive, symmetric, transitive).
It partitions the domain into these equivalence classes: {1, 2}, {3}, {4}

☒ **R is not a Partial Order (POSET).**

MATRIX REPRESENTATION

R	1	2	3	4
1	1	1	0	0
2	1	1	0	0
3	0	0	1	0
4	0	0	0	1

Entry $M[i][j] = 1$ when $(i, j) \in R$, otherwise 0. Rows are the first element, columns the second; the shaded diagonal entries are the loops.

Figure 5: An equivalence relation with its equivalence classes and relation matrix

Figure 6 uses the *divides* relation on $\{1, 2, 3, 4\}$. It is reflexive, antisymmetric and transitive, so it is a **partial order (POSET)**. It is not symmetric; the missing mirror pairs such as $(2, 1)$ are listed.

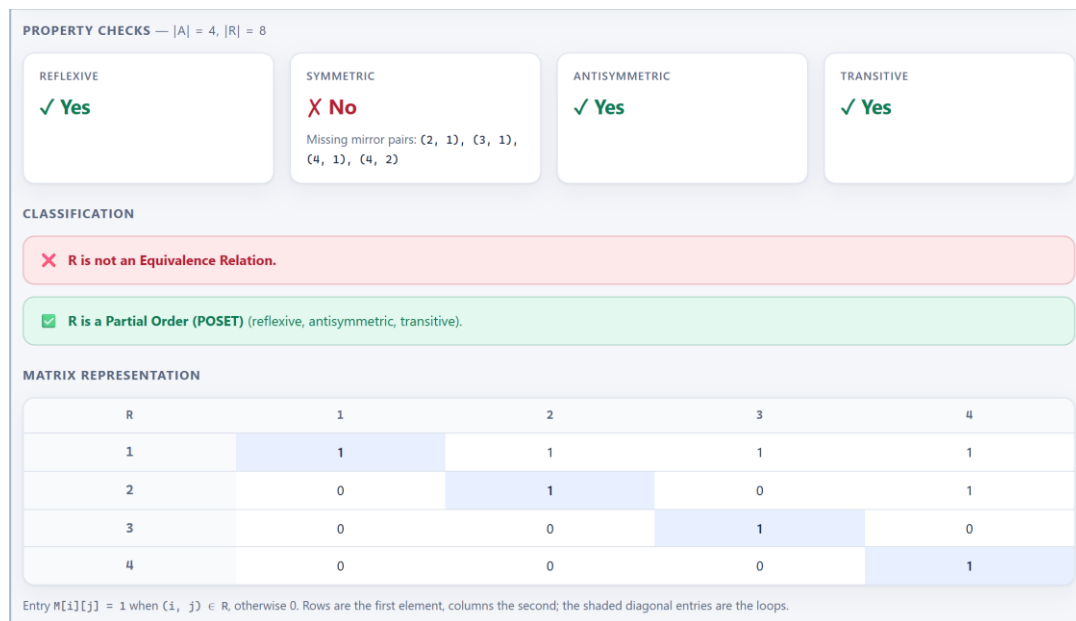


Figure 6: The divides relation identified as a partial order (POSET)

Finally, Figure 7 shows a relation that is neither. For $R = \{(a, b), (b, c)\}$ on $\{a, b, c\}$, the toolkit lists the missing loops, the missing mirror pairs and the pair (a, c) that is required for transitivity.

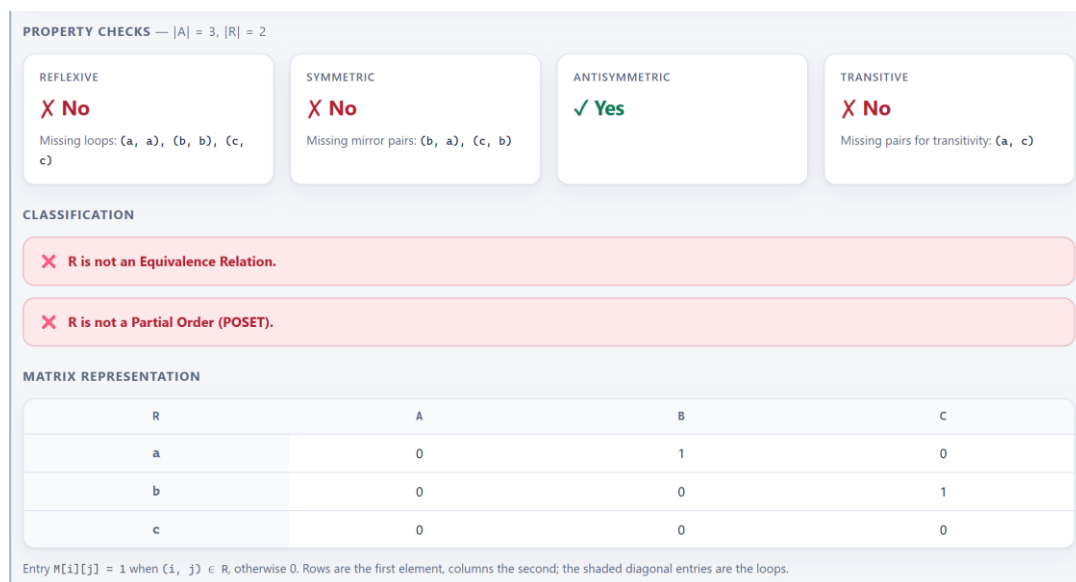


Figure 7: A relation that is neither an equivalence relation nor a partial order, with every failure explained

5.3 Graph Algorithm Visualizer

The graph module is tested with the following weighted graph of six vertices and nine edges:

Table 7: Sample weighted graph used in the results

Edge	A–B	A–C	B–C	B–D	C–D	C–E	D–E	D–F	E–F
Weight	4	2	1	5	8	10	2	6	3

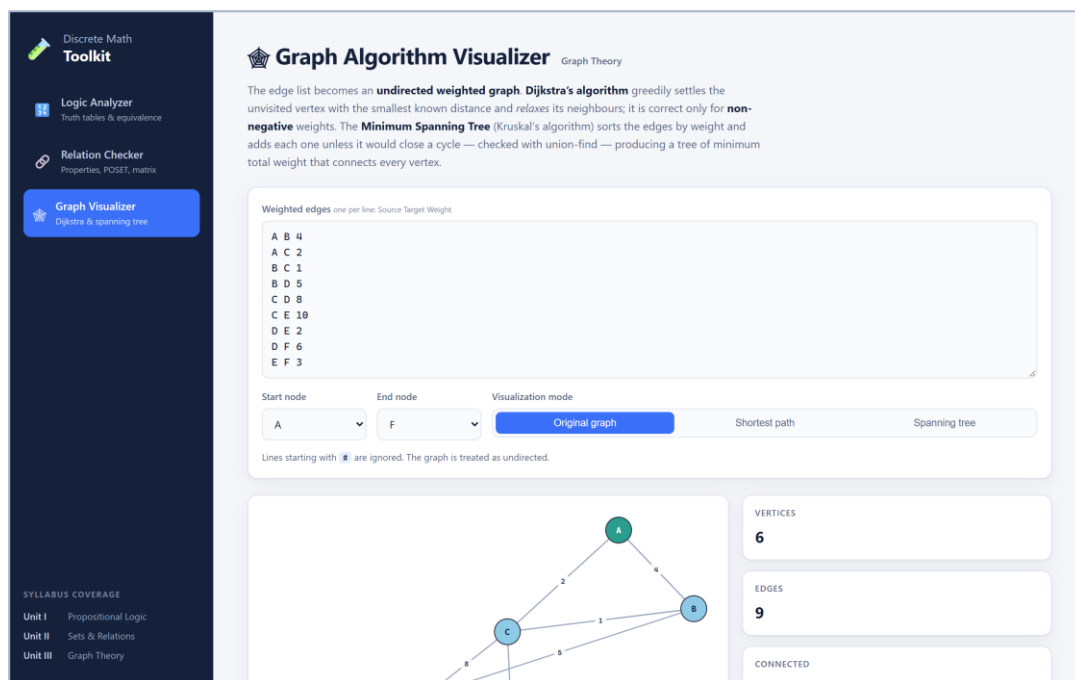


Figure 8: The Graph Algorithm Visualizer with the edge list and controls

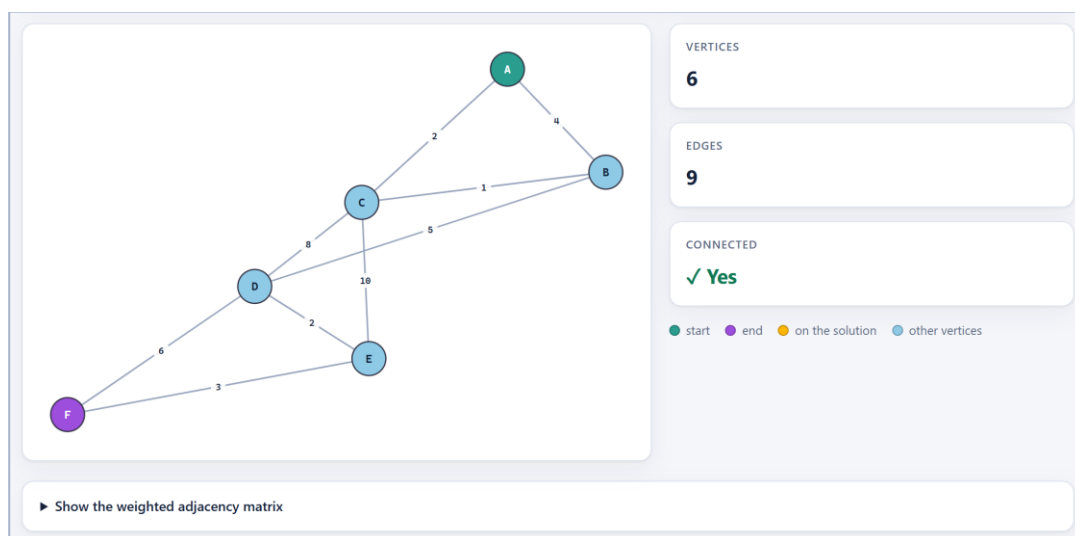


Figure 9: The input graph drawn with a force-directed layout

Shortest path from A to F. Dijkstra's algorithm finds the path $A \rightarrow C \rightarrow B \rightarrow D \rightarrow E \rightarrow F$ with total cost $2 + 1 + 5 + 2 + 3 = 13$ (Figure 10). Note that the direct-looking route $A \rightarrow B$

costs 4, whereas $A \rightarrow C \rightarrow B$ costs only 3, so the algorithm correctly prefers the longer route through C.

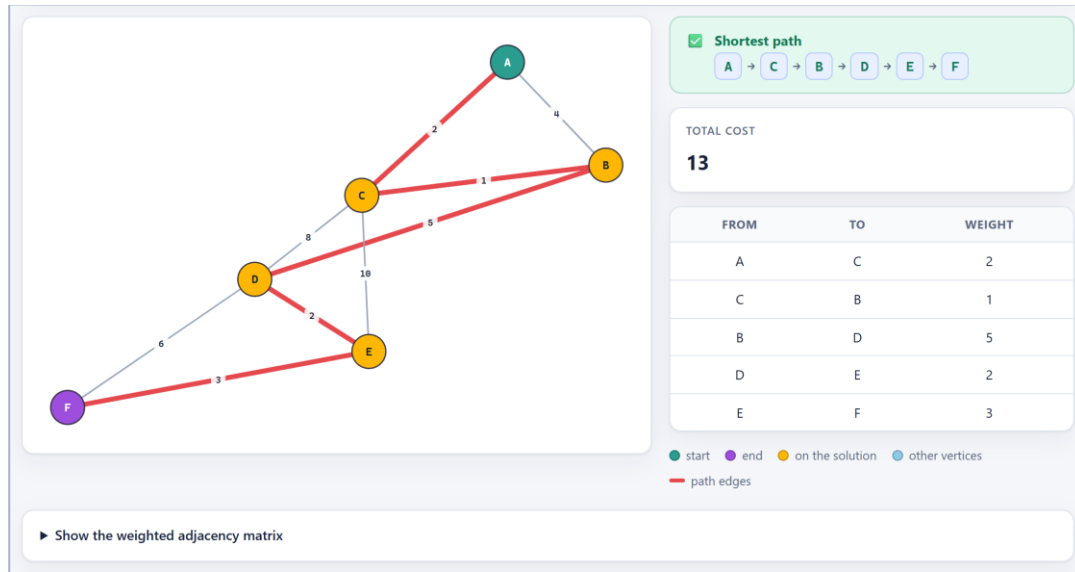


Figure 10: Shortest path from A to F found by Dijkstra's algorithm (cost 13)

Minimum spanning tree. Kruskal's algorithm considers the edges in increasing order of weight: B–C (1), A–C (2), D–E (2), E–F (3) are accepted; A–B (4) is rejected because A and B are already connected; B–D (5) is accepted, after which all six vertices are connected. The tree has 5 of the 9 edges and total weight 13 (Figure 11).

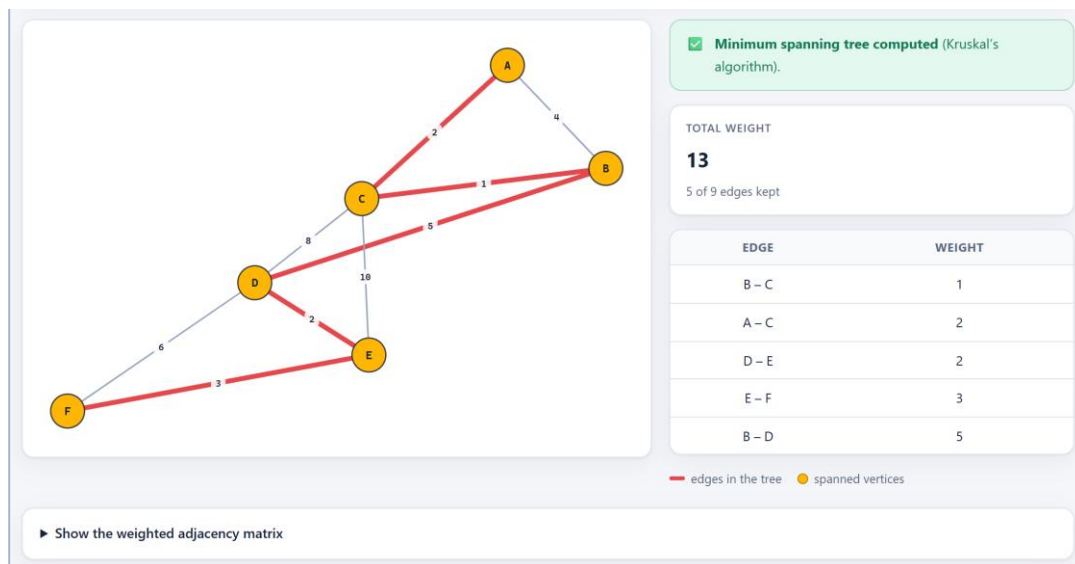


Figure 11: Minimum spanning tree found by Kruskal's algorithm (total weight 13)

5.4 Verification of Results by Hand

Each result shown above was checked against a manual calculation:

Table 8: Manual verification of the toolkit's outputs

Test case	Expected (by hand)	Toolkit output	Status
$p \rightarrow q$ vs $\neg p \vee q$	Equivalent (implication law)	Logically equivalent	Correct
$p \rightarrow q$ vs $q \rightarrow p$	Not equivalent; differ at $p=T, q=F$	Not equivalent, same counter-example	Correct
$\sim(p \ \& \ q)$ vs $\sim p \mid \sim q$	Equivalent (De Morgan)	Logically equivalent	Correct
Equivalence example	Classes $\{1,2\}, \{3\}, \{4\}$	Classes $\{1,2\}, \{3\}, \{4\}$	Correct
Divides on $\{1,2,3,4\}$	POSET, not symmetric	POSET, not symmetric	Correct
Shortest path $A \rightarrow F$	A–C–B–D–E–F, cost 13	A–C–B–D–E–F, cost 13	Correct
Minimum spanning tree	5 edges, weight 13	5 edges, weight 13	Correct

6. TESTING

The Node.js version includes an automated test suite written with Node's built-in test runner (`node:test`). It is run with a single command, `npm test`, and needs no extra packages.

Table 9: Automated test suite

Test file	Tests	What is tested
test/logic.test.js	14	Tokenizer and parser, operator precedence, truth tables, tautology / contradiction / contingency, DNF and CNF minimisation, equivalence laws, error handling
test/relations.test.js	12	Parsing of domains and pairs, each of the four properties with its witnesses, equivalence classes, POSET detection, relation matrix, empty relation
test/graph.test.js	12	Edge-list parsing and error messages, self-loops and duplicate edges, Dijkstra (cheapest route rather than fewest hops, unreachable targets), Kruskal MST and spanning forests, adjacency matrix, deterministic layout
test/api.test.js	14	End-to-end tests against a running server: all three endpoints, invalid input, static files, and protection against path-traversal requests
Total	52	All 52 tests pass

Beyond the automated tests, the interface was checked in both light and dark themes and at phone width. This testing found and fixed a layout bug in which the sidebar made narrow pages scroll sideways by 12 pixels.

7. HOW TO RUN THE PROJECT

The project runs entirely on the local machine and does not need an internet connection.

7.1 Node.js Version (Recommended)

1. Install **Node.js 18 or newer** from nodejs.org. No other software or npm packages are required.
2. Open the project folder and double-click **start.bat** (Windows) or run **./start.sh** (macOS/Linux). The browser opens automatically.
3. Alternatively, open a terminal in the project folder and run **npm start**, then visit <http://localhost:3000>.
4. To run the test suite, use **npm test**.

<code>npm start</code>	start the server on http://localhost:3000
<code>node server.js --open</code>	start the server and open the browser
<code>PORT=4000 npm start</code>	use a different port
<code>npm run dev</code>	restart automatically when a file changes
<code>npm test</code>	run all 52 tests

If port 3000 is already in use, the server automatically moves to the next free port and prints the address it used.

7.2 Python Version (Prototype)

```
python -m pip install -r requirements.txt
python -m streamlit run app.py           # opens http://localhost:8501
```

8. CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The Discrete Math Toolkit meets all of its objectives. It solves standard problems from three units of the MAT2002 syllabus — logical equivalence and normal forms, properties of binary relations, and shortest paths and spanning trees in weighted graphs — and, just as importantly, it *explains* each answer through counter-examples, witness pairs and highlighted drawings.

Implementing the algorithms from first principles, instead of calling library functions, was the most valuable part of the project. Writing a parser required a precise understanding of operator precedence; implementing Quine–McCluskey made the structure of normal forms concrete; and implementing Dijkstra and Kruskal with a heap and a union-find structure showed how the theory of greedy algorithms becomes working code. The final application is safe, dependency-free, verified by 52 passing tests, and starts with a single click.

8.2 Future Scope

- **Step-by-step animation** of Dijkstra's and Kruskal's algorithms, showing each settled vertex or accepted edge in turn.
- **Hasse diagrams** for partial orders, together with maximal, minimal, greatest and least elements and lattice checks.
- **Closures** of relations (reflexive, symmetric and transitive closure via Warshall's algorithm).
- **More graph algorithms:** Prim's MST, BFS/DFS traversal, Euler and Hamiltonian paths, graph colouring and directed graphs.
- **Predicate logic** and inference rules, with step-by-step proofs of validity.
- Exporting results (truth tables, matrices and graph drawings) as PDF or images for assignments.

REFERENCES

- [1] K. H. Rosen, *Discrete Mathematics and Its Applications*, 8th ed. New York: McGraw-Hill Education, 2019.
- [2] J. P. Tremblay and R. Manohar, *Discrete Mathematical Structures with Applications to Computer Science*. New Delhi: Tata McGraw-Hill, 1997.
- [3] N. Deo, *Graph Theory with Applications to Engineering and Computer Science*. New Delhi: PHI Learning, 1974.
- [4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA: MIT Press, 2022.
- [5] E. W. Dijkstra, "A note on two problems in connexion with graphs," *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [6] J. B. Kruskal, "On the shortest spanning subtree of a graph and the traveling salesman problem," *Proceedings of the American Mathematical Society*, vol. 7, no. 1, pp. 48–50, 1956.
- [7] E. J. McCluskey, "Minimization of Boolean functions," *Bell System Technical Journal*, vol. 35, no. 6, pp. 1417–1444, 1956.
- [8] Node.js Documentation, OpenJS Foundation. [Online]. Available: <https://nodejs.org/docs/>